

Erasmus Mundus Joint Master Artificial Intelligence for
Sustainable Societies (AISS)

Topics on Machine Learning and its Applications

Model training

Pedro Strecht



Model training

Introduction

Classification problems

Data partitioning

Prediction models

Learning algorithms for classification

Learning algorithms for regression

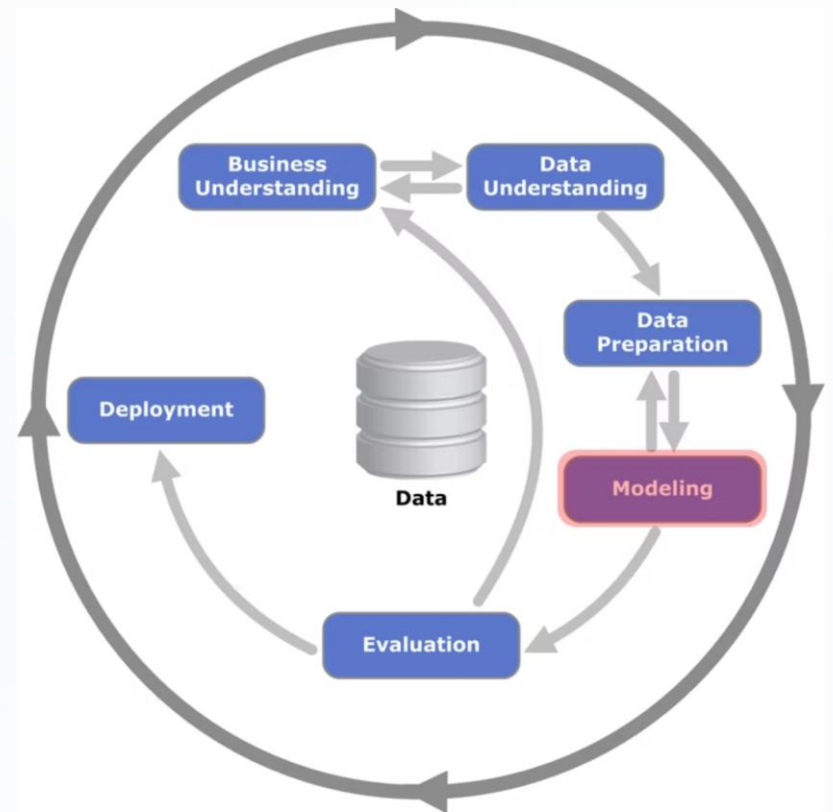
Case studies of model training

Model training

Introduction

In CRISP-DM **Model training** corresponds to **Modeling**:

- The **modeling phase** involves selecting, applying, and optimizing various modeling techniques to the prepared dataset
- Since different algorithms excel in different scenarios, **multiple techniques are explored** to identify the most effective approach for the specific data mining problem
- Each technique may have unique requirements, sometimes necessitating revisiting the data preparation phase for adjustments
- It involves **experimenting with diverse algorithms**, fine-tuning parameters, and evaluating performance to achieve optimal results



Model training

Introduction

ML is commonly divided into two major areas each encompassing a wide range of algorithms and techniques.



Supervised learning

Make predictions about new data related to a specific phenomenon



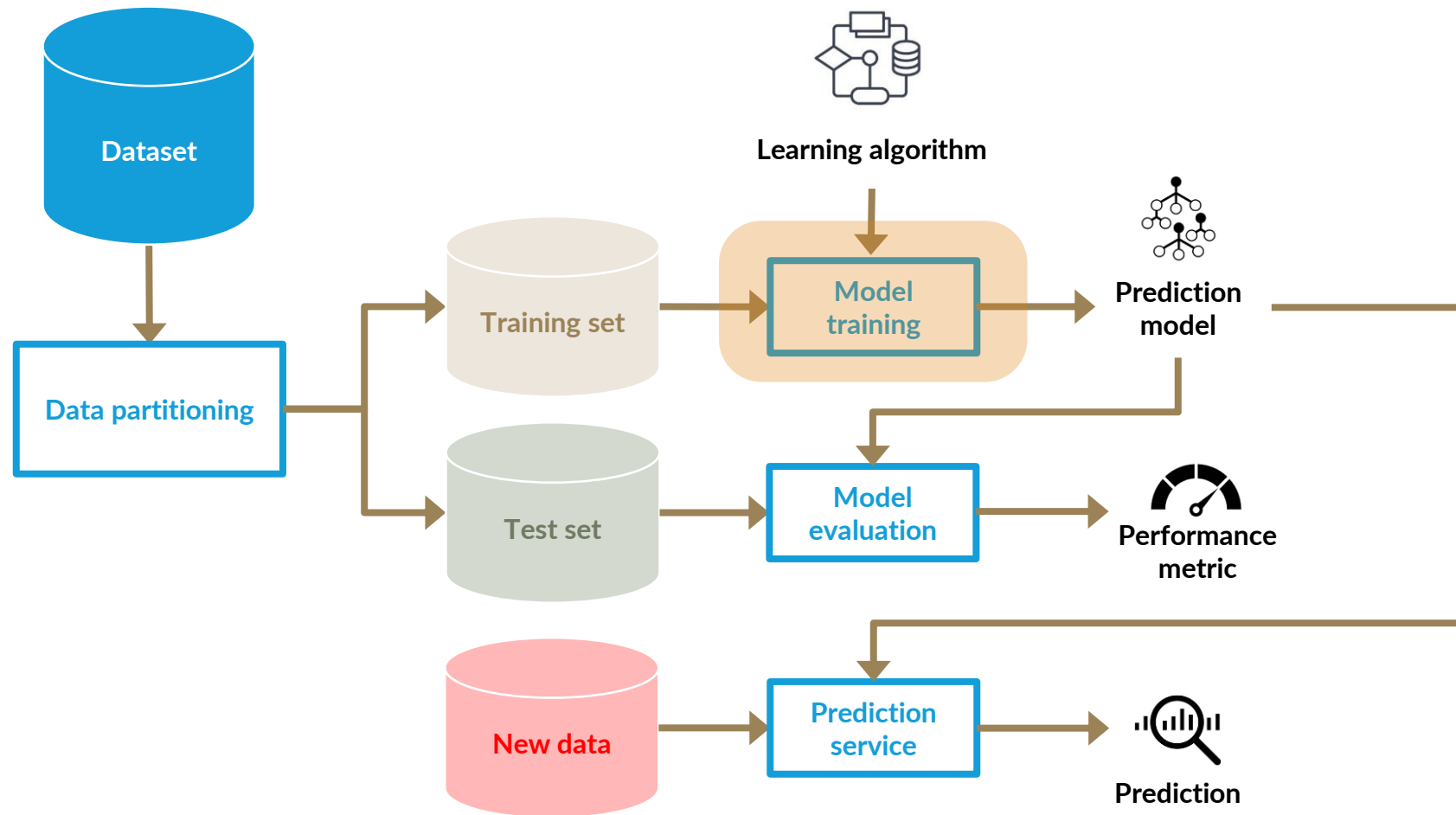
Unsupervised learning

Reveal relationships and patterns within the data that otherwise might not be immediately apparent

Model training

Introduction

General experimental set-up



Model training

Introduction

- **Classification** addresses prediction problems by assigning examples to predefined classes, corresponding to categories
 - A classification model could be employed to determine, for example:
 - whether a student will pass or fail a course
 - assess a customer's likelihood of churning (canceling as a service)
- Also known as *classifiers*, these types of models learn from datasets where each example is associated with a known class
- Through this process, the patterns and relationships between independent variables and the corresponding classes of a target variable are discovered and assembled in a **prediction model**

Model training

Introduction

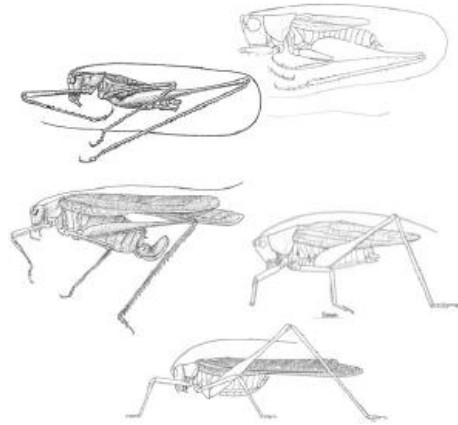
- **Regression** focuses on predicting continuous-valued numbers
 - For example, a regression model could be used to:
 - predict a student's final exam mark or
 - estimate the sales of a particular product over a period of time
- These prediction models, also referred to as *regressors*, extract patterns and relationships between the independent variables and the corresponding target values
- Like classifiers, regressors can predict target values for new examples

Model training

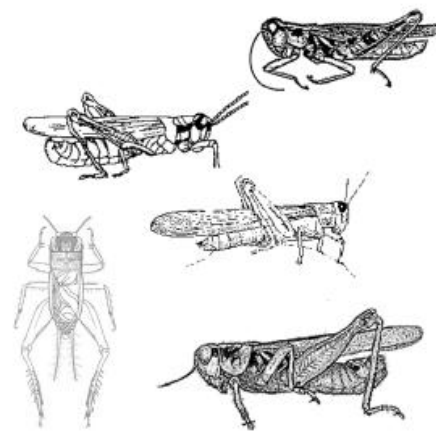
Classification problems

- A set of classified insects is given: **five bush crickets** and **five grasshoppers**

Bush crickets



Grasshoppers

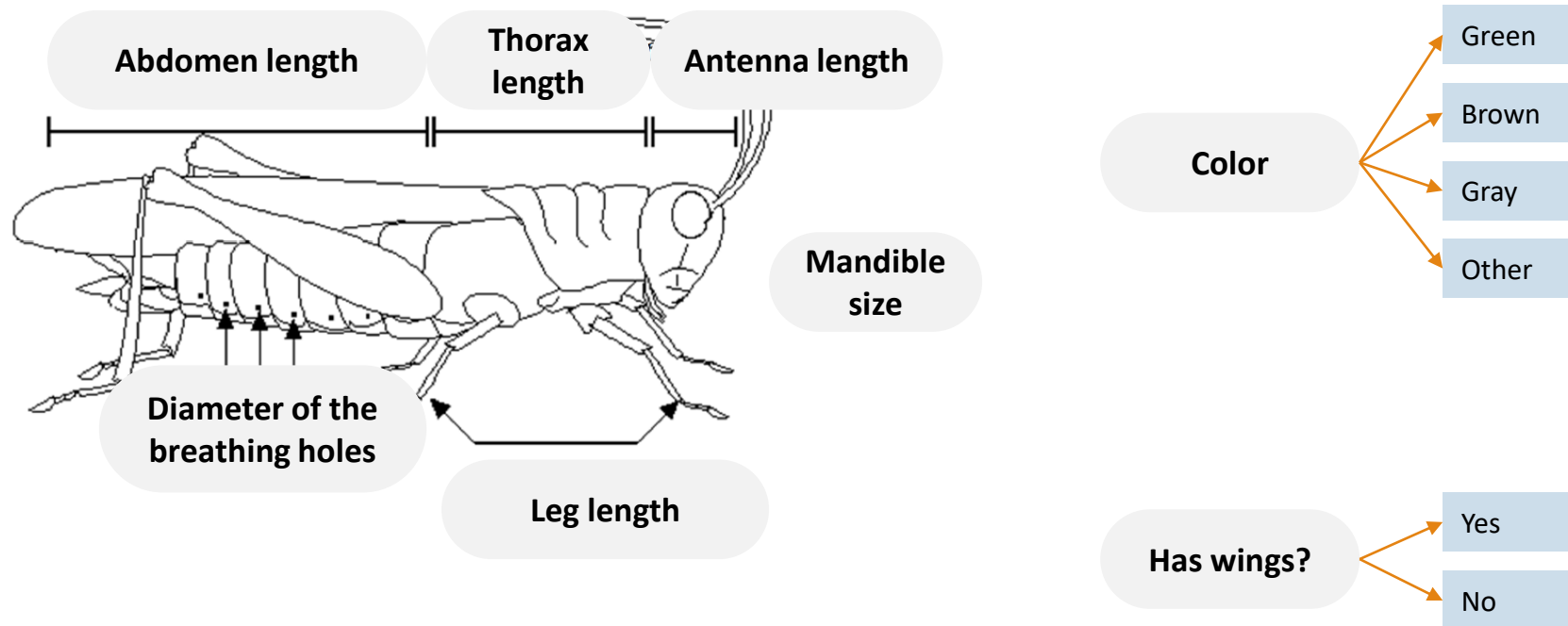


- The bush cricket is an insect similar to a grasshopper and is found in North America
- The goal is to classify the type of insect for a new specimen

Model training

Classification problems

- In any field of knowledge, we can measure characteristics (features):



Model training

Classification problems

- The characteristics can be stored in a dataset
- The classification problem can now be expressed as:
 - Given a training set predict the **class** of a **new example**

Insect ID	Abdomen length	Antenna length	Type of insect
1	2.7	5.5	Grasshoper
2	8.0	9.1	Bush crickets
3	0.9	4.7	Grasshoper
4	1.1	3.1	Grasshoper
5	5.4	8.5	Bush crickets
6	2.9	1.9	Grasshoper
7	6.1	6.6	Bush crickets
8	0.5	1.0	Grasshoper
9	8.3	6.6	Bush crickets
10	8.1	4.7	Bush crickets



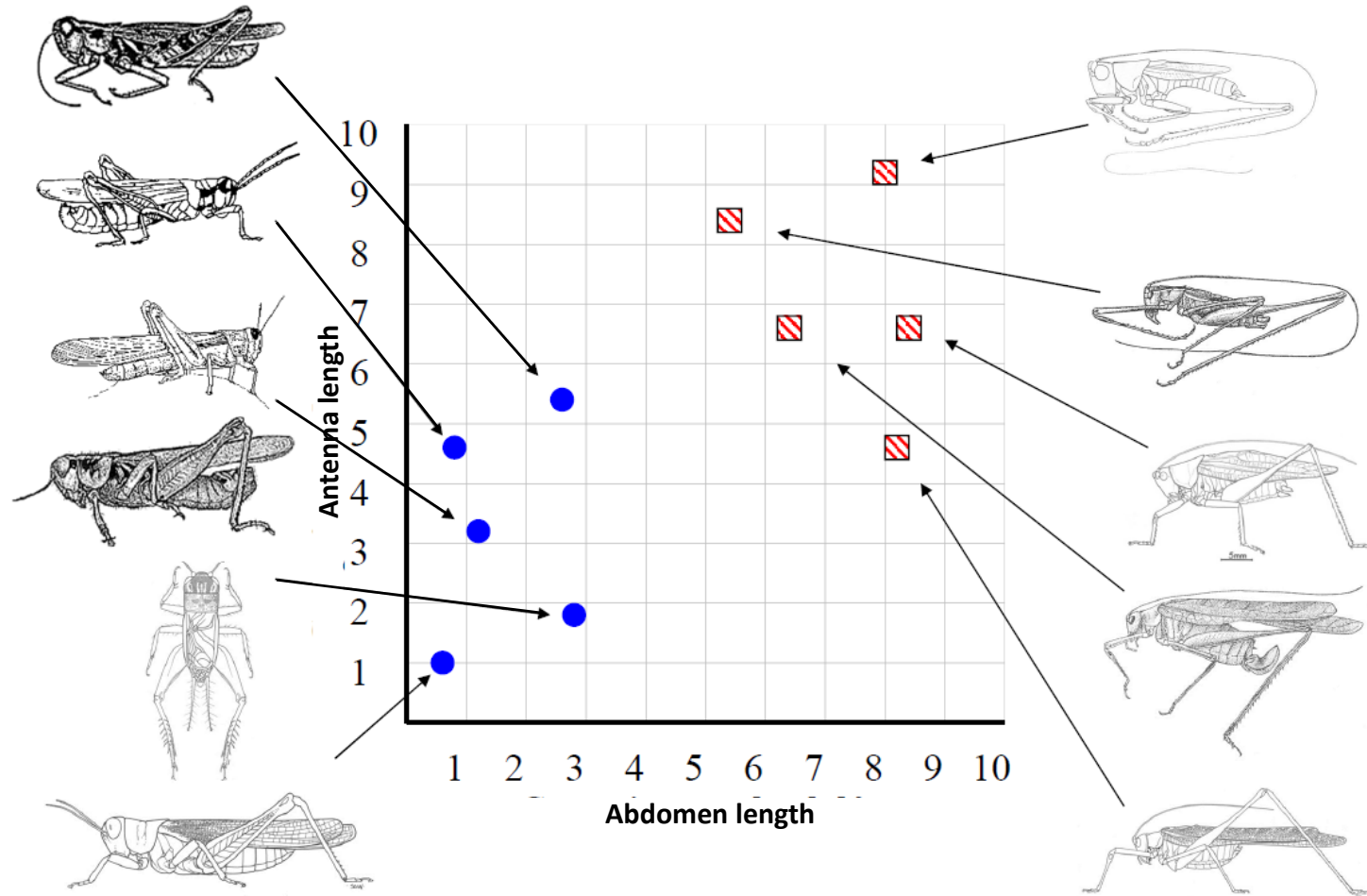
11	5.1	7.0	???
----	-----	-----	-----

Model training

Classification problems

Grasshoppers

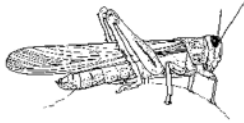
Bush crickets



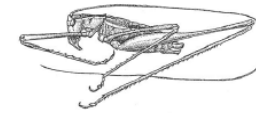
Model training

Classification problems

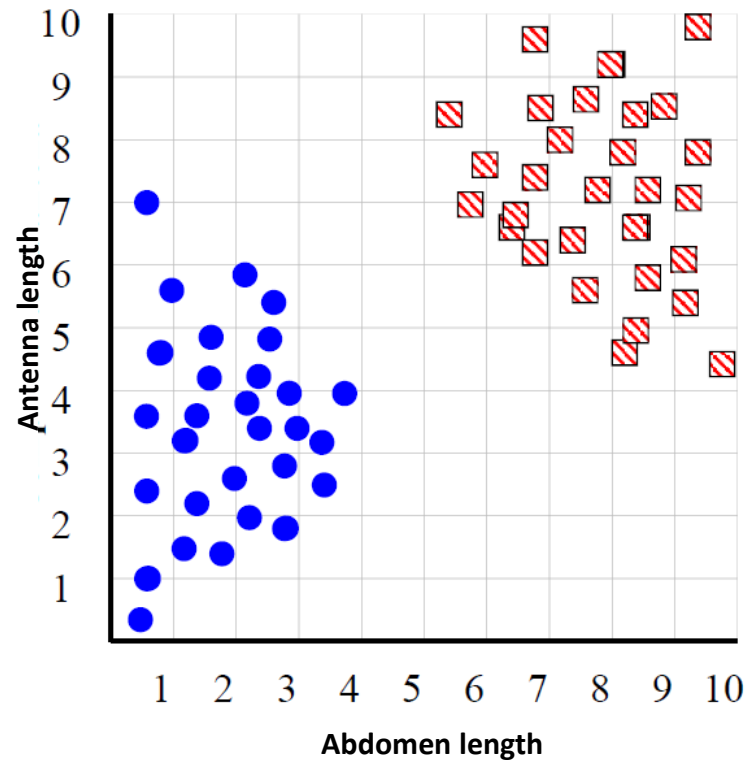
Grasshoppers



Bush crickets



Larger set



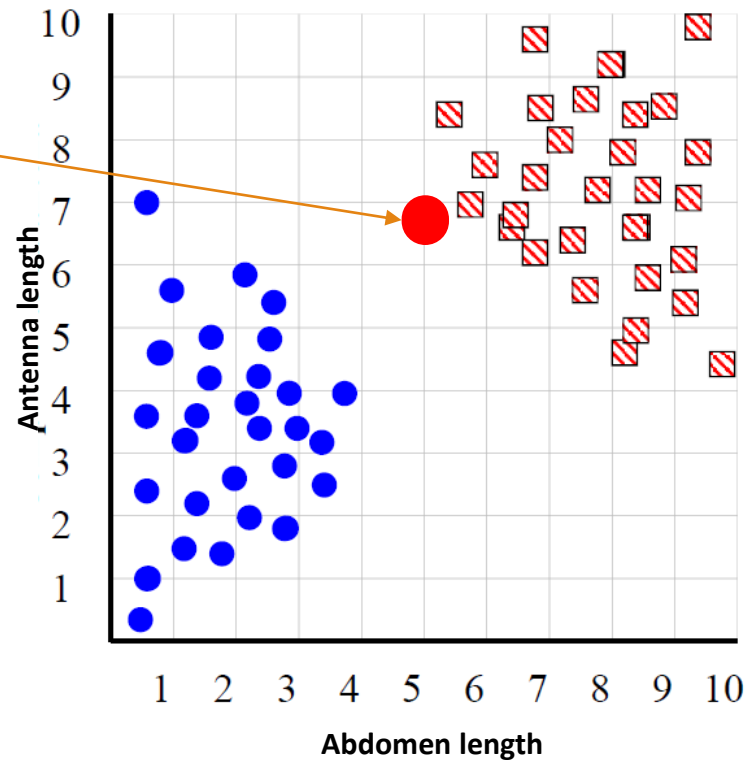
Model training

Classification problems

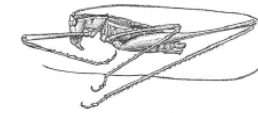
Grasshoppers



Larger set



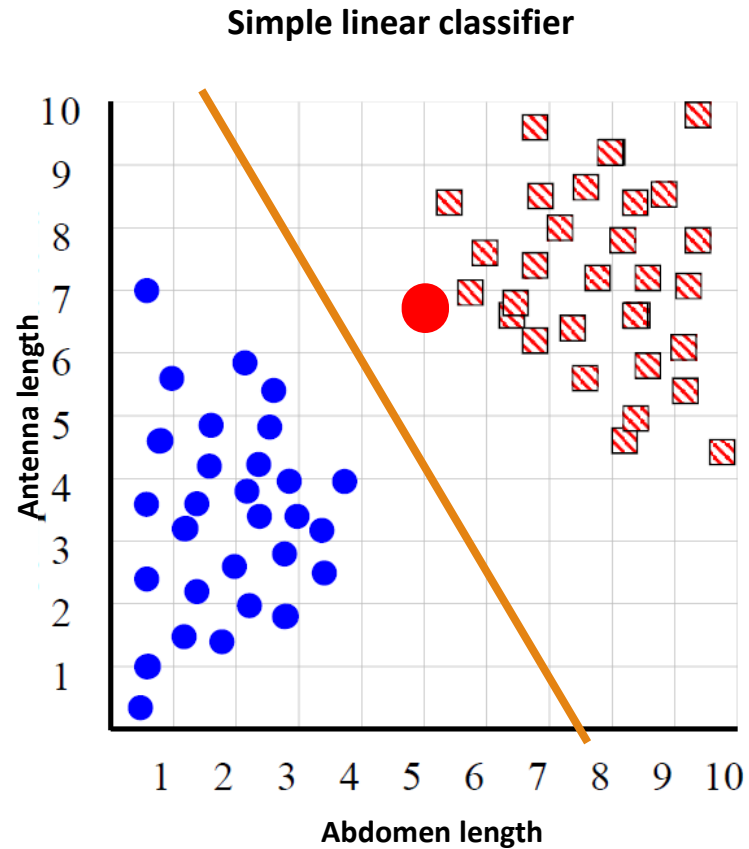
Bush crickets



- The unclassified example can be *projected* into the same space as the database
- Details of the problem can be abstracted
- It is much easier to consider as points in space

Model training

Classification problems

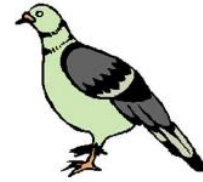


- If the unclassified example is above the line:
 - Then the class is **Bush Cricket**
 - Otherwise, the class is **Grasshopper**

Model training

Classification problems

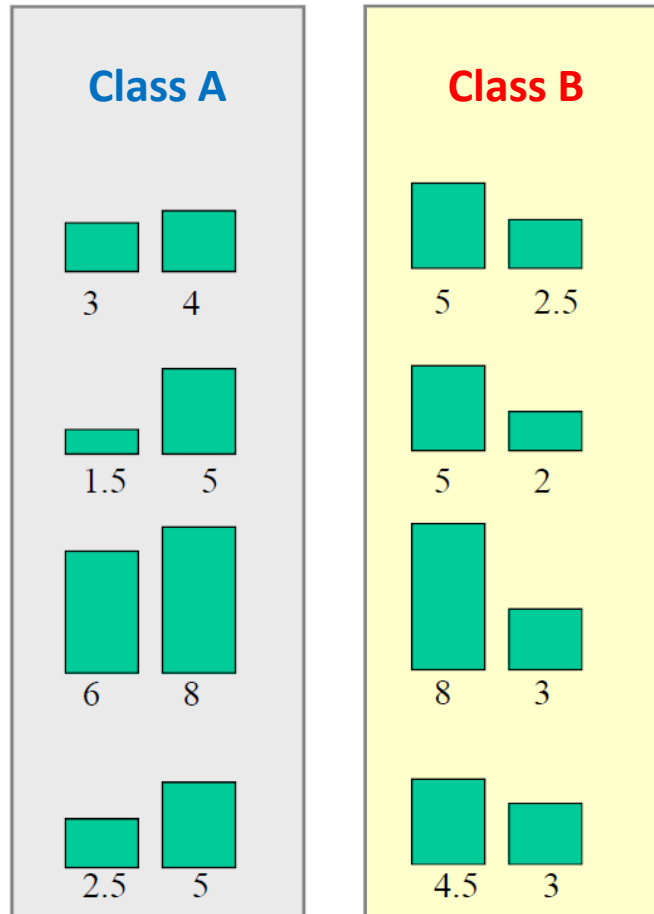
- A few problems that were shown to pigeons
- are called "Pigeonhole Problems"
- Let's test them with humans...



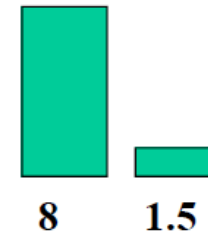
Model training

Classification problems

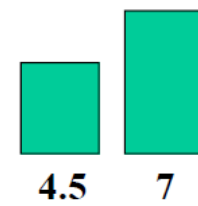
■ 1st problem



What is the class of this object?



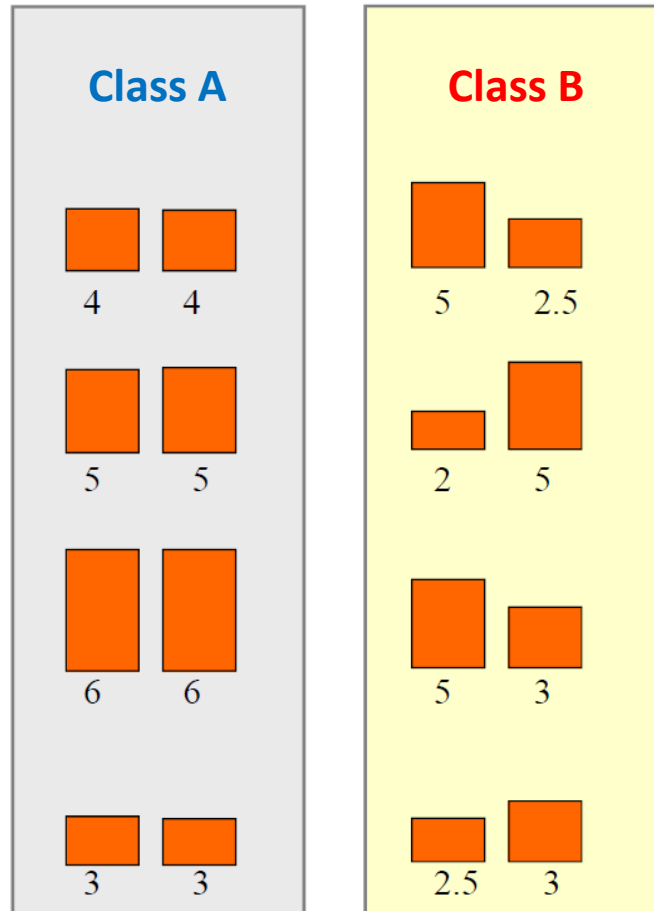
And this one?



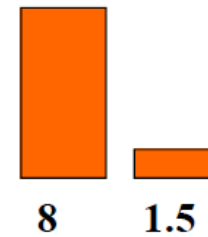
Model training

Classification problems

■ 2nd problem



What is the class of this object?



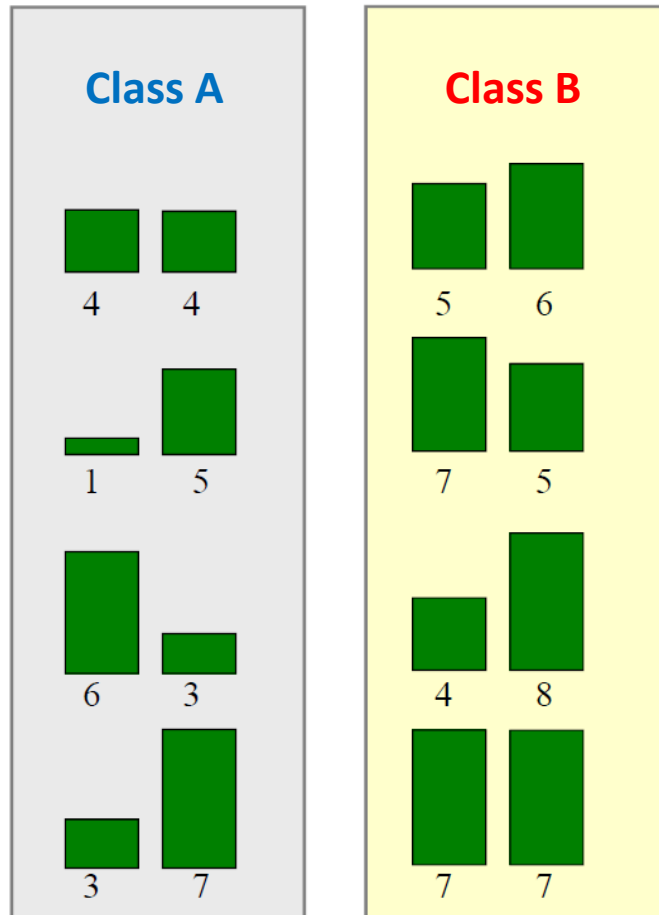
And this one?



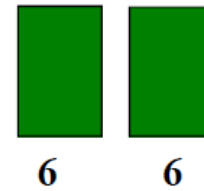
Model training

Classification problems

■ 3rd problem



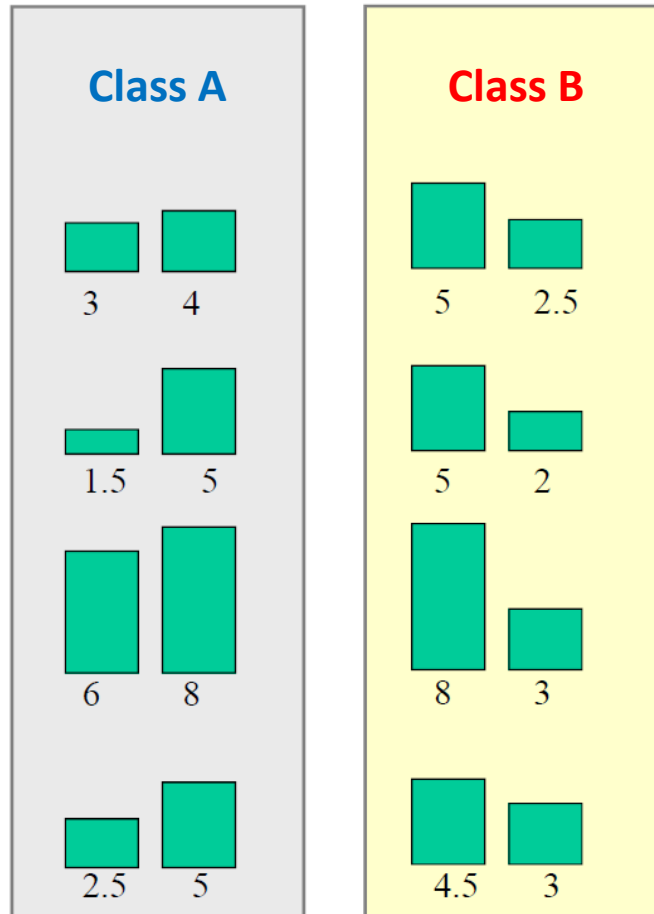
What is the class of this object?



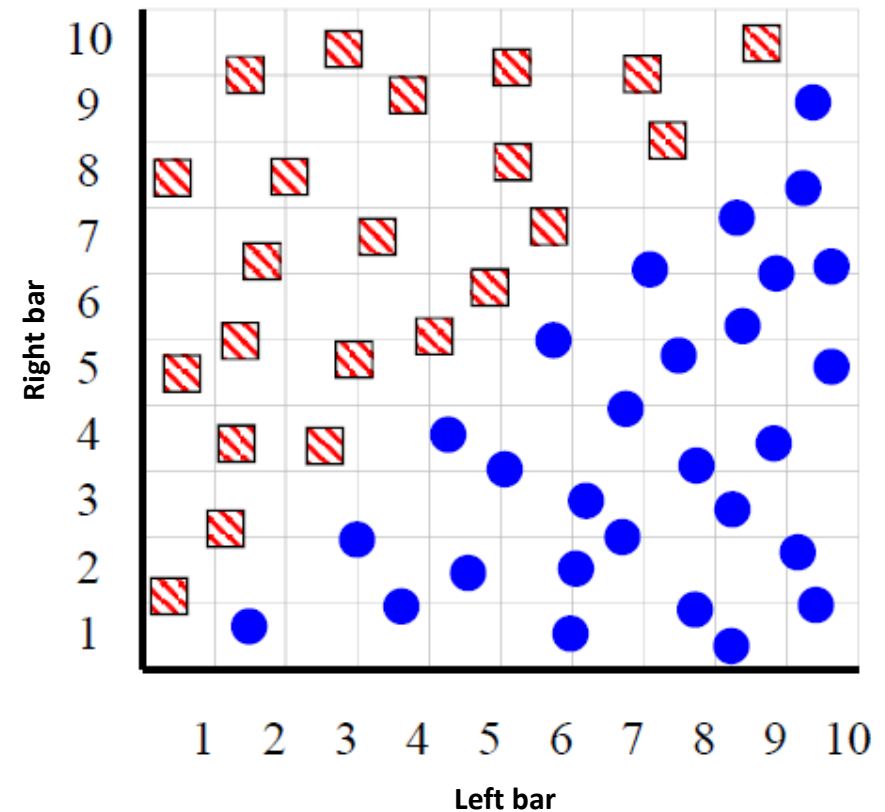
Model training

Classification problems

■ 1st problem



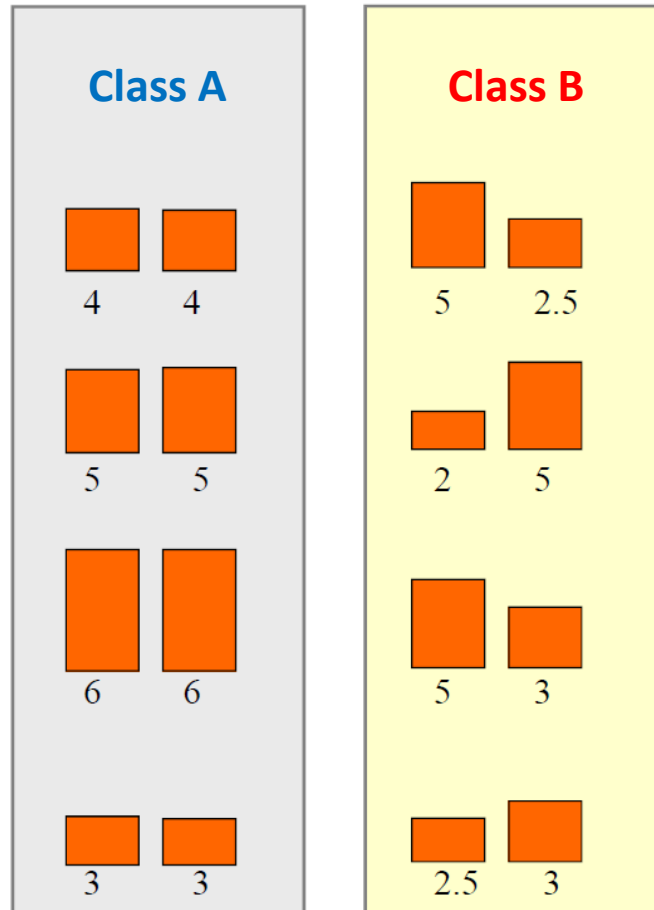
The rule is if the left bar is smaller than the right, it is **A**;
otherwise, it is **B**



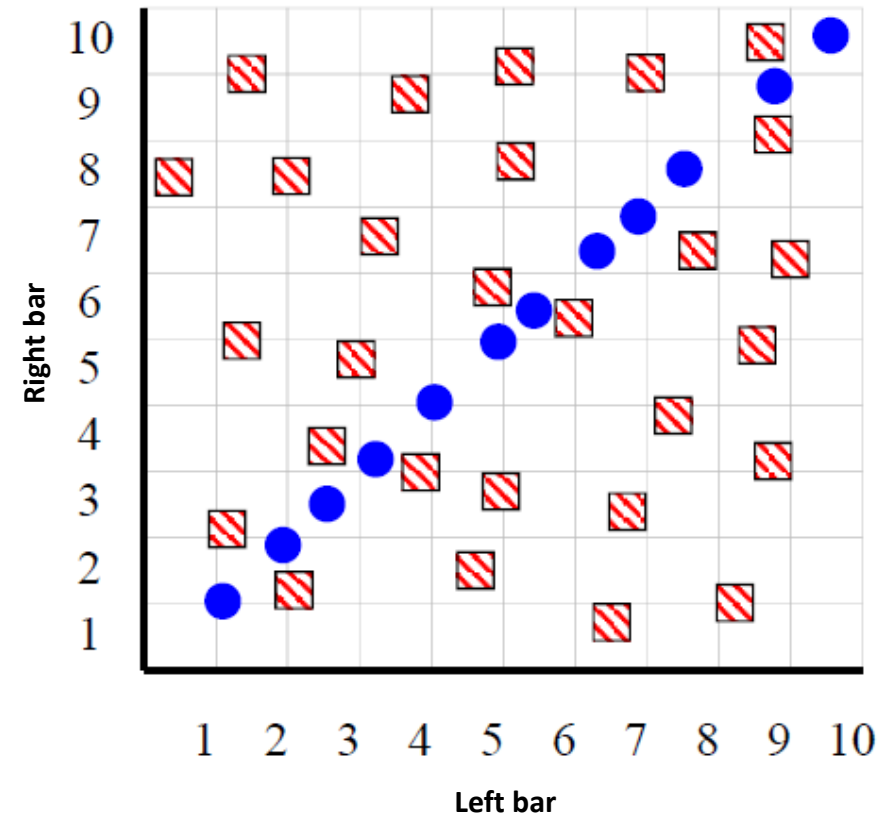
Model training

Classification problems

■ 2nd problem



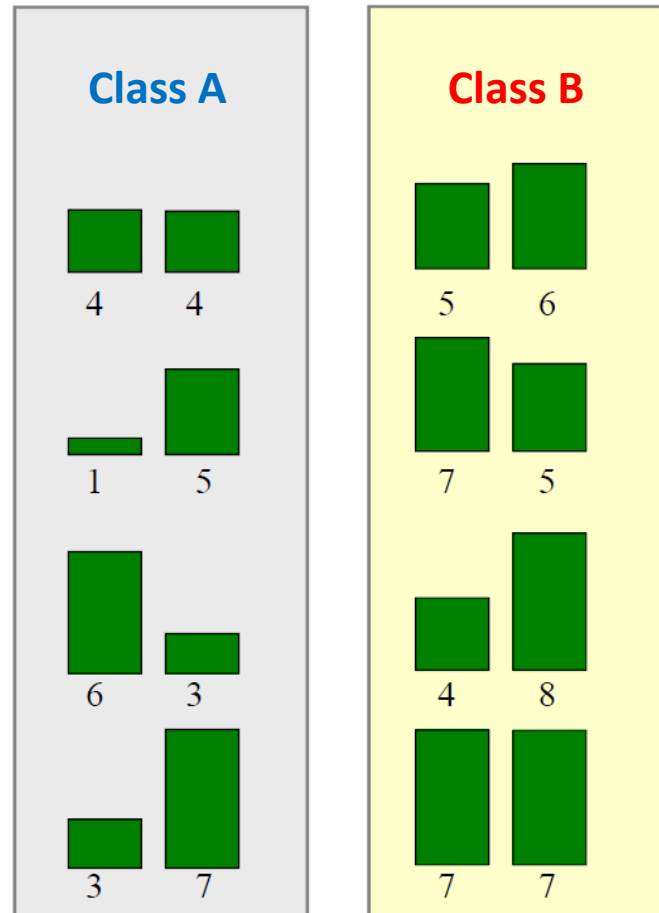
The rule is if the two bars are of equal size, it is **A**;
otherwise, it is **B**



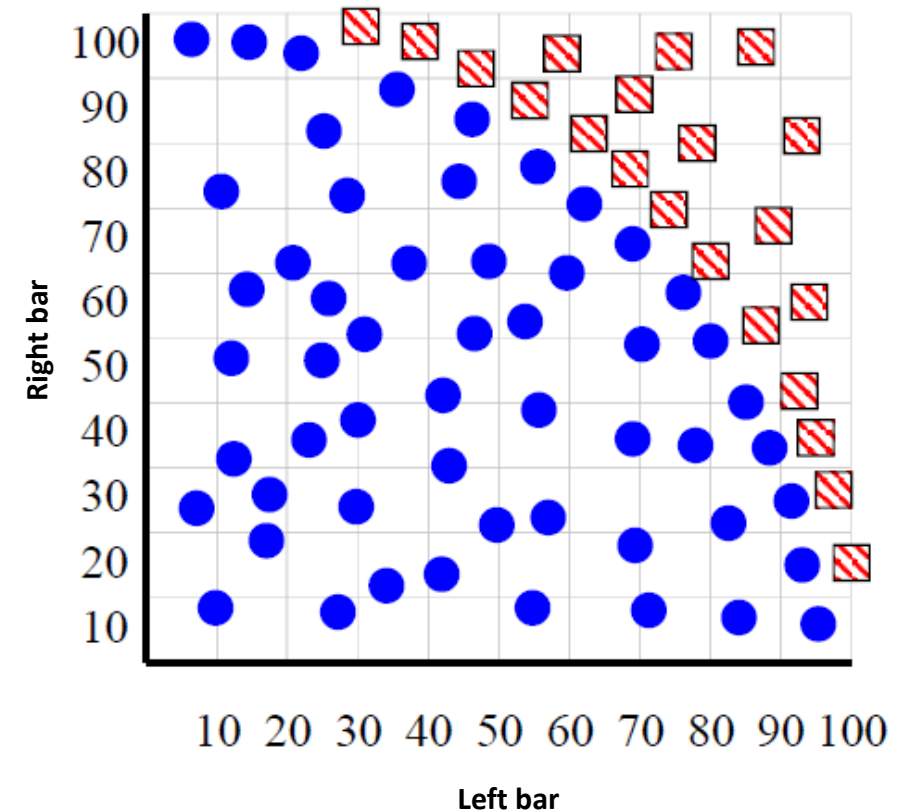
Model training

Classification problems

■ 3rd problem



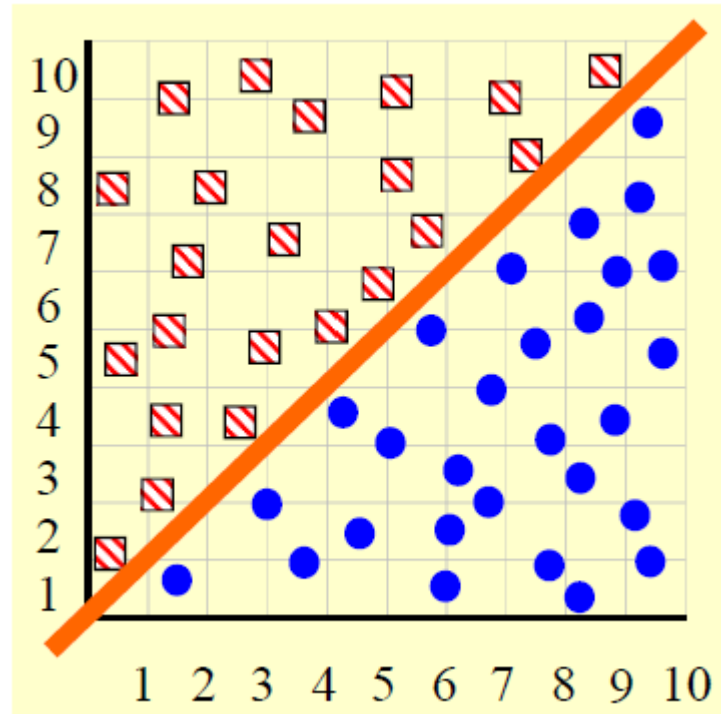
The rule is if the square of the sum of the two bars is less than or equal to 100, it is **A**; otherwise, it is **B**



Model training

Classification problems

- Which of the "Pigeonhole Problems" can be solved with the Simple Linear Classifier?
 - **Perfect**

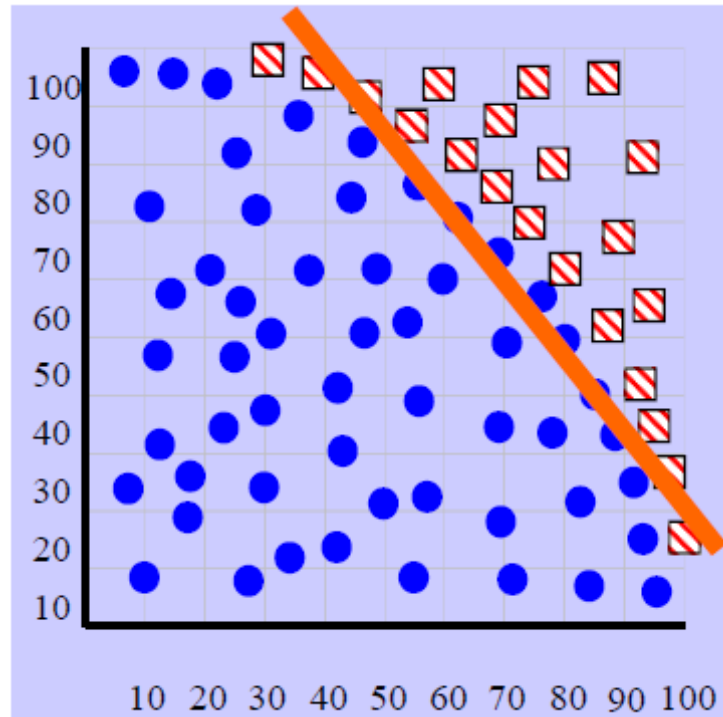


Model training

Classification problems

- Which of the "Pigeonhole Problems" can be solved with the Simple Linear Classifier?

- **Reasonable**

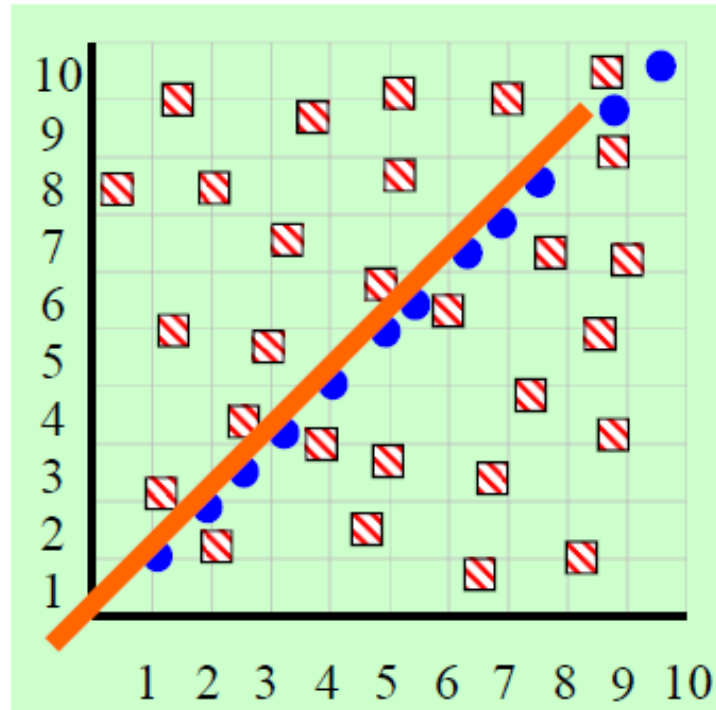


Model training

Classification problems

- Which of the "Pigeonhole Problems" can be solved with the Simple Linear Classifier?

- **Useless**



Model training

Classification problems

- A famous problem from R. A. Fisher is available on the **Iris dataset** (lilies)

- 3 classes (multiclass problem)



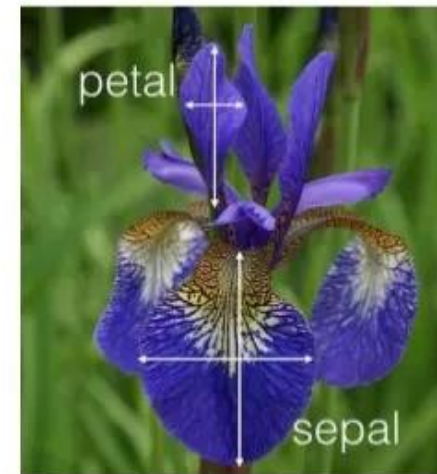
Iris Setosa



Iris Versicolor



Iris Virginica



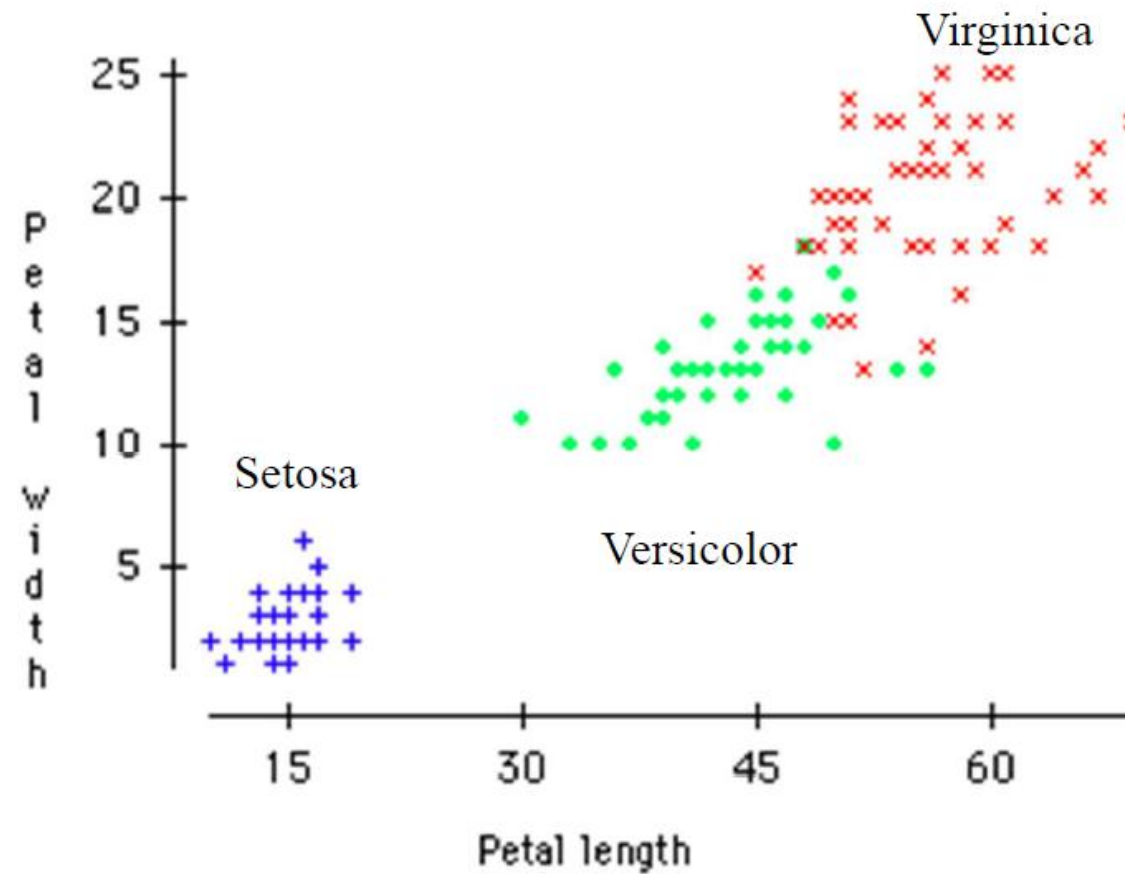
- 50 examples of each class
- The task is to classify the variety of lilies into one of the 3 existing varieties using:
 - Sepal Length
 - Sepal Width
 - Petal Length
 - Petal Width

Training / test data				
Features				Labels
Sepal length	Sepal width	Petal length	Petal width	Species
5.1	3.5	1.4	0.2	Iris setosa
4.9	3.0	1.4	0.2	Iris setosa
7.0	3.2	4.7	1.4	Iris versicolor
6.4	3.2	4.5	1.5	Iris versicolor
6.3	3.3	6.0	2.5	Iris virginica
5.8	3.3	6.0	2.5	Iris virginica

Model training

Classification problems

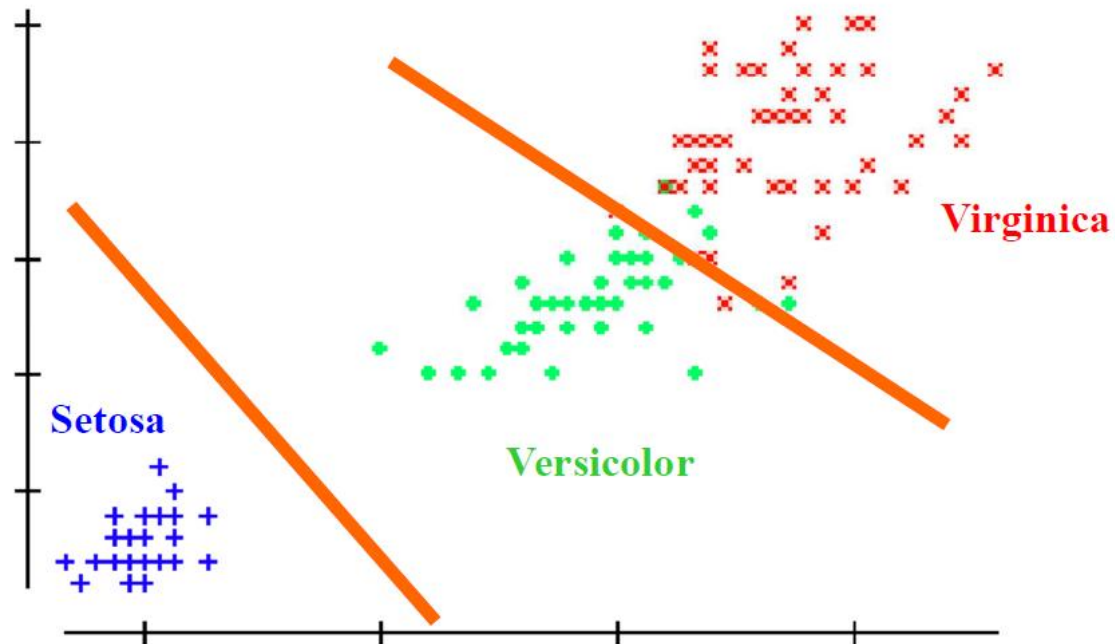
- Iris dataset



Model training

Classification problems

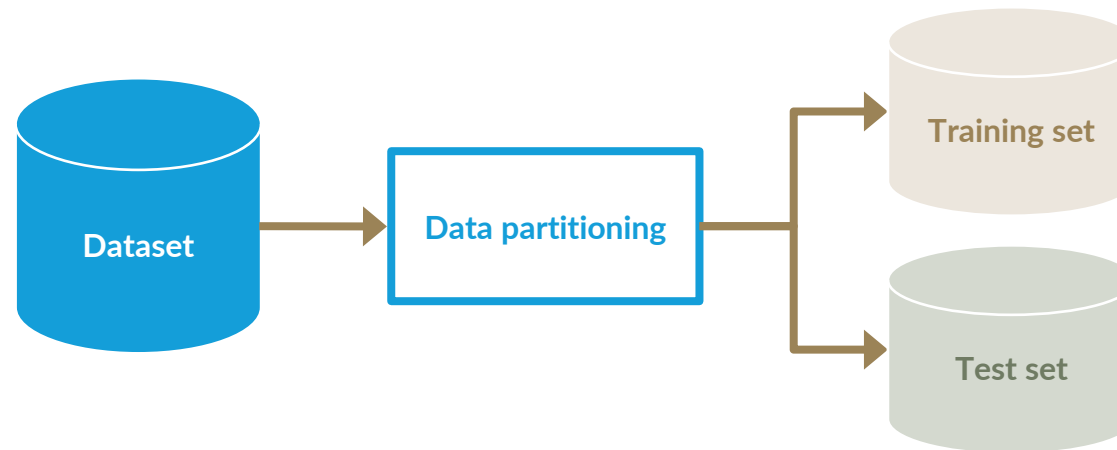
- The linear classifier can be generalized for N classes by fitting $N - 1$ lines
 - In this case, the first line is fitted to perfectly discriminate between Setosa and Virginica/Versicolor, and then the focus shifts to approximately discriminating between Virginica and Versicolor



Model training

Data partitioning

- **Datasets** form the foundation for developing and evaluating predictive models
- Typically, a dataset is partitioned into two subsets:
 - a **training set**, where the model learns from the data
 - a **test set**, where the model's performance is evaluated



Model training

Data partitioning

- A **training set**, denoted as Tr , is a dataset used to train a prediction model
- Regarding the variables, the set of **independent variables**, denoted as X_{Tr} , is treated separately from the **target variable**, denoted as $\{y\}$, which the model predicts

$$X_{Tr} = \{x_1, x_2, \dots, x_d\}$$

- The complete set of variables V_{Tr} encompasses both, with d , in this context, denoting the **dimensionality** of the training set, i.e., considering only the number of independent variables ($d = |X_{Tr}|$)

$$V_{Tr} = X_{Tr} \cup \{y\}$$

Model training

Data partitioning

- An **example** within a training set includes values for both independent variables ($v_{i,j}$) and the target variable (y_i)

$$e_i = (v_{i,1}, v_{i,2}, \dots, v_{i,d}, y_i)$$

- A training set can be structured in a **tabular format**, with columns representing the variables and rows representing individual examples

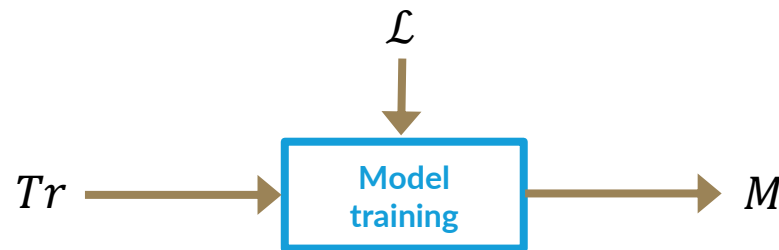
	x_1	x_2	$\dots$	x_d	y
e_1	$v_{1,1}$	$v_{1,2}$	$\dots$	$v_{1,d}$	y_1
e_2	$v_{2,1}$	$v_{2,2}$	$\dots$	$v_{2,d}$	y_2
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
e_n	$v_{n,1}$	$v_{n,2}$	$\dots$	$v_{n,d}$	y_n

- In this context, n denotes the **size** of the training set ($n = |Tr|$)

Model training

Predictions models

- A **prediction model**, often abbreviated as *model* and denoted as M , is the outcome of analyzing a dataset to formulate a **mapping function** between the values of independent variables and the corresponding values of the target variable across all examples
- A **learning algorithm** or *learner* is any procedure or function used to build a model, denoted as $\mathcal{L}$



Model training

Predictions models

- The function that uses:
 - a learning algorithm ($\mathcal{L}$)
 - a training set (Tr)
 - the set of independent variables (X_{Tr})
 - and the target variable (y)

to create a prediction model (M) is usually referred to as **Train**

$$M = \text{Train}(\mathcal{L}, Tr, X_{Tr}, y)$$

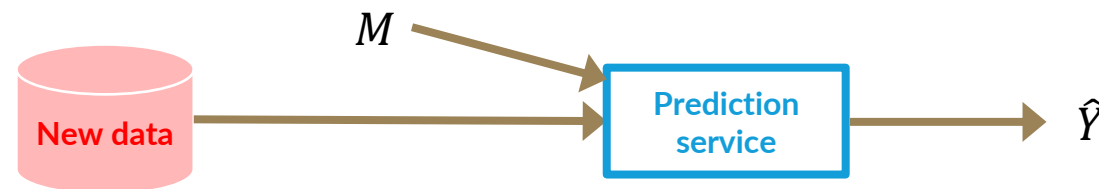
Model training

Predictions models

- The function that uses a prediction model M to predict the target variable for a given example e is commonly referred to as **Predict**

$$\hat{y} = \text{Predict}(M, e)$$

- The result of this operation is a **prediction**, denoted as $\hat{y}$
- After training, a model (M) can be applied to predict the values of the target variable ($\hat{y}$) for new examples, typically where the target variable is unknown, also referred to as *unlabeled examples*



Model training

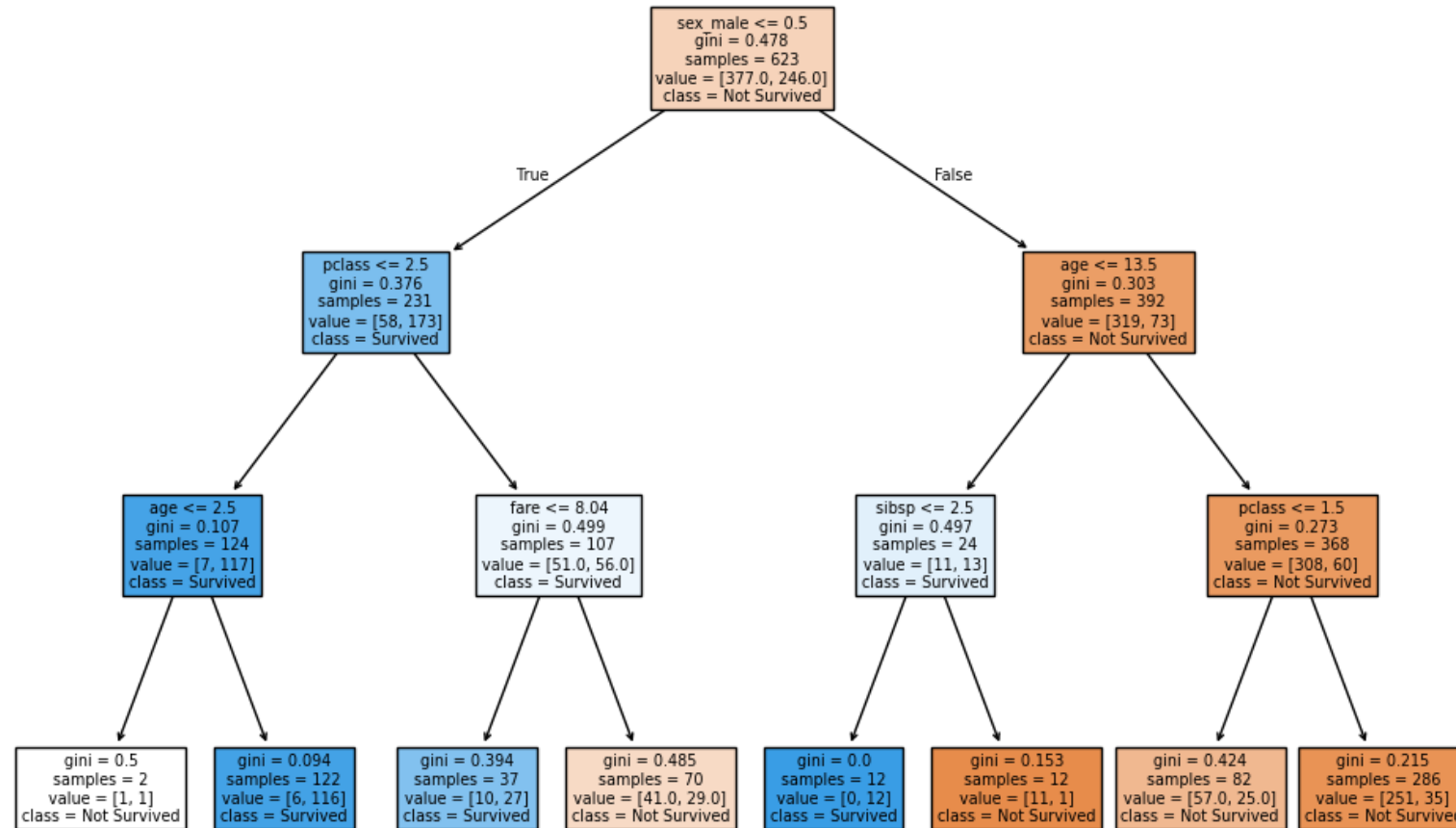
Predictions models

- The term **predictor** is highly context-dependent, with its definition shifting based on the specific context
 - In some contexts, a *predictor* is a synonym of a prediction model (Breiman, 1996; Mendes-Moreira et al., 2012)
 - However, in other contexts, a *predictor* refers to an independent variable within a model, also known as a *predictor variable*, with the target variable being referred to as the *response variable* (Green, 1978; Han et al., 2012; Mizumoto, 2023)

Model training

Predictions models

- Example of a prediction model: **Decision tree**



Model training

Learning algorithms

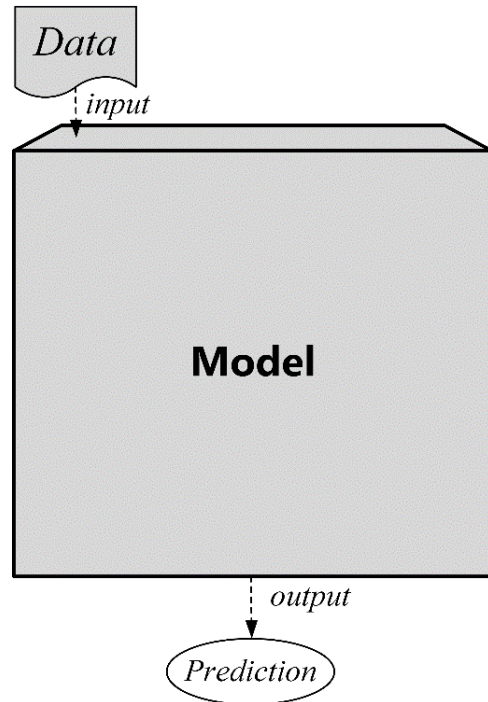
- Both classification and regression encompass a large set of **learning algorithms** with diverse approaches and underlying principles, providing flexibility in selecting the most suitable method based on the specific characteristics of the dataset and the problem at hand
- There is currently a **very large number** of learning algorithms and several more are being created
- According to how they work, they can be grouped into categories:
 - Distance-based algorithms
 - Probability-based algorithms
 - Search based algorithms
 - Optimization-based algorithms

Model training

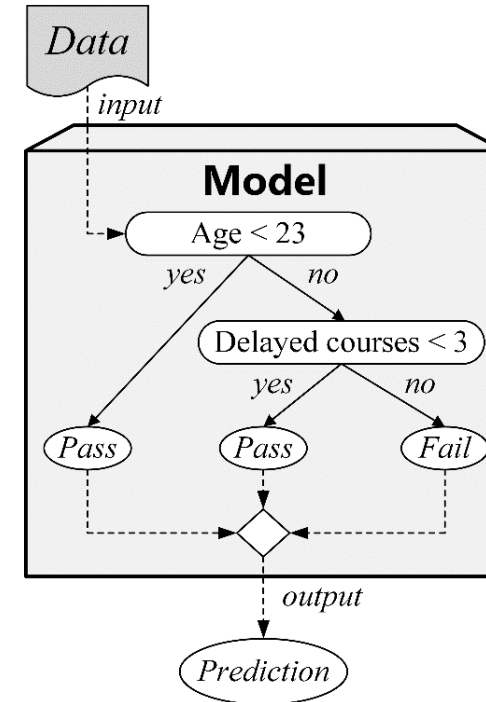
Learning algorithms

- Usually learning algorithms are also categorized relating the **interpretability** of the models

Black-box models



White-box models



Model training

Learning algorithms for classification

- **Distance-based algorithms** predict the class of new objects by looking at the class of similar objects
- They are also known as *lazy algorithms*
- They do not explicitly create model
- The most well-known is **k-nearest neighbor** (kNN)
 - Considered the simplest classification algorithm
 - Predict the class of a new example as the most frequent class among the *nearest* examples

Model training

Learning algorithms for classification

- **Probability-based algorithms** assume a probabilistic relationship between the independent variables and the target variable
- **Discriminative Models**
 - Learn the posterior distribution $P(Y|X)$
 - Directly estimate class probabilities given X
 - Example: **Logistic Regression**
- **Generative Models**
 - Learn the joint distribution $P(X, Y)$
 - Derive $P(Y|X)$ using the Bayes' theorem.
 - Example: **Naïve Bayes**

Model training

Learning algorithms for classification

- **Search-based algorithms** explore the decision space to identify models with high interpretability by discovering explicit decision rules
- **Decision tree induction algorithms**
 - Recursively partition the input space
 - Produce tree-structured models composed of decision rules (e.g., **C5.0** and **CART**)
 - Interpretable and easy to visualize
- **Model tree induction algorithms**
 - Similar structure to decision trees
 - Leaf nodes contain predictive models (e.g., linear models) instead of simple class labels
 - Combine interpretability with increased predictive flexibility (e.g. **MARS**)

Model training

Learning algorithms for classification

- **Optimization-based algorithms** rely on a strong mathematical foundation by learning a predictive model by optimizing a predictive function to minimize predictive error
- **Parametrized models**
 - Minimize a loss function over many parameters
 - Can handle a very high dimensional decision space
 - Produce very complex models with high non-linear relationships (e.g., **Neural Networks**)
- **Margin based optimization**
 - Maximize the margin between classes of the target variable
 - If necessary, project the decision space into higher dimensions to make classes linearly separable
 - Produce complex models with hyperplane separation of classes (e.g. **Support Vector Machines**)

Model training

Learning algorithms for classification

Algorithm Category	Examples of algorithms	Predictive performance of models	Interpretability of models	Key idea
Distance-based algorithms	k-Nearest Neighbors (kNN)	Moderate	Medium	Classify based on closest instances; no explicit model
Probability-based algorithms	Logistic Regression, Naïve Bayes	Good	Medium to High	Learn probabilistic relationships between features and target
Search-based algorithms	Decision Trees (CART, C5.0), Model Trees, MARS	Moderate	Medium to Very High	Explicit rule discovery; trees easy to visualize
Optimization-based algorithms	Neural Networks, SVM	High to Very High	Very Low	Optimize complex functions; handle non-linear, high-dimensional patterns

Model training

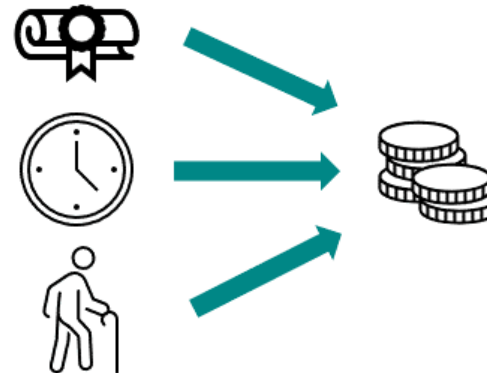
Learning algorithms for regression

- Linear regression analysis models the relationship between a dependent variable and one or more independent variables
 - **Simple linear regression** – there is only one independent variable
 - **Multiple linear regression** – there are two or more independent variables

Simple Linear Regression



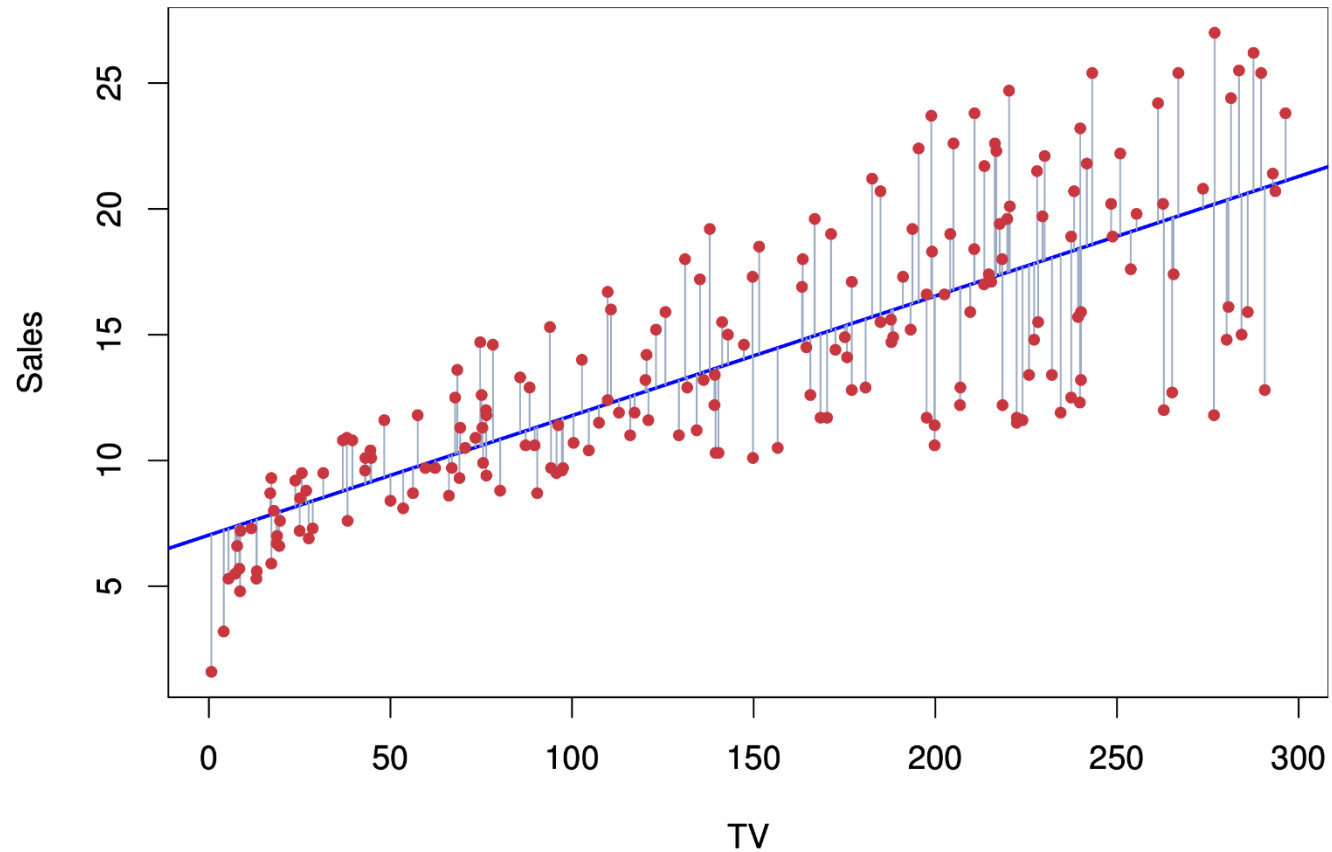
Multiple Linear Regression



Model training

Learning algorithms for regression

- **Simple linear regression** aims to find the **regression line** that best represents the linear relationship between the dependent and independent variable



Model training

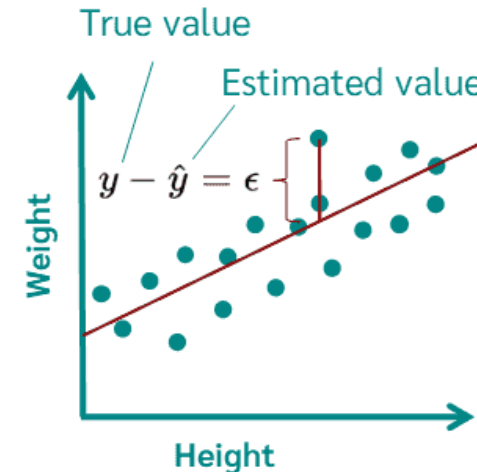
Learning algorithms for regression

- The **regression line** can be described by the following equation

$$\hat{y} = b \cdot x + a$$

Estimated dependent variable Slope Independent variable y intercept

- Where a and b are the **regression coefficients**
 - a – **intercept**, the point of intersection with the y-axis
 - b – **slope**, the gradient of the straight line
- $\hat{y}$ is the **estimate** of the y -value
- The objective is to minimize the estimation error, ensuring that the difference between the predicted and actual values remains small
- This difference, known as the **residual**, or error, is denoted by ϵ (epsilon)



$$y = b \cdot x + a + \epsilon$$

Model training

Learning algorithms for regression

- **Multiple linear regression** aims to find the **regression hyperplane** that best represents the linear relationship between the dependent and independent variables

Simple Linear
Regression

$$\hat{y} = b \cdot x + a$$



Multiple Linear
Regression

$$\hat{y} = b_1 \cdot x_1 + b_2 \cdot x_2 + \dots + b_k \cdot x_k + a$$

- The coefficients in multiple linear regression are interpreted similarly to those in simple linear regression:
 - The intercept a represents the predicted value of the dependent variable when **all independent variables are zero**
 - Each coefficient b indicates the expected change in the dependent variable y when the corresponding independent variable x **increases by one unit**, assuming all other variables remain constant

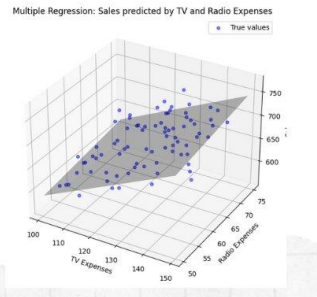
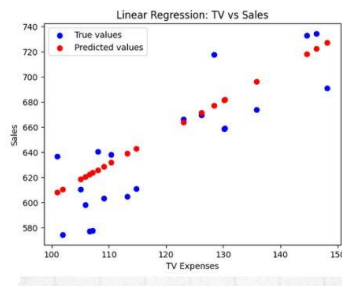
Model training

Learning algorithms for regression

- **Multiple regression** examines the effect of several independent variables on a single dependent variable

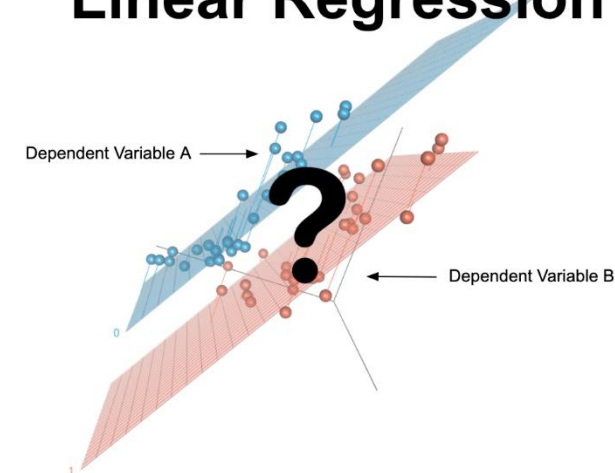
- **Multivariate regression** involves multiple dependent variables, requiring separate regression models to analyze their relationships

LINEAR REGRESSION VS MULTIPLE REGRESSION



Difference Between Linear and Multiple Regression:
<https://www.shiksha.com/online-courses/articles/linear-and-multiple-regression/>

Multivariate Multiple Linear Regression

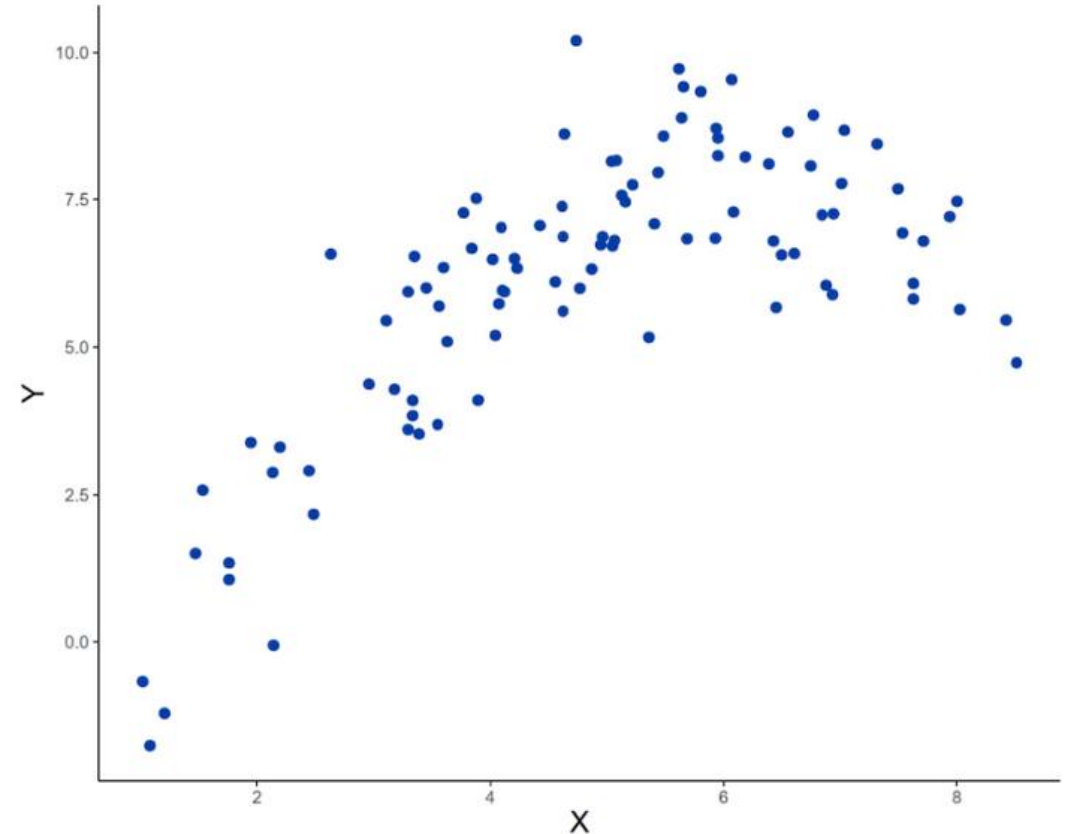


Multivariate Multiple Linear Regression:
<https://www.statstest.com/multivariate-multiple-linear-regression/>

Model training

Learning algorithms for regression

- A simple linear regression algorithm only works when the relationship between the data is linear
- If the relationship data is non-linear, then linear regression will not be able to draw a best-fit line



Polynomial Regression: An Introduction: <https://builtin.com/machine-learning/polynomial-regression>

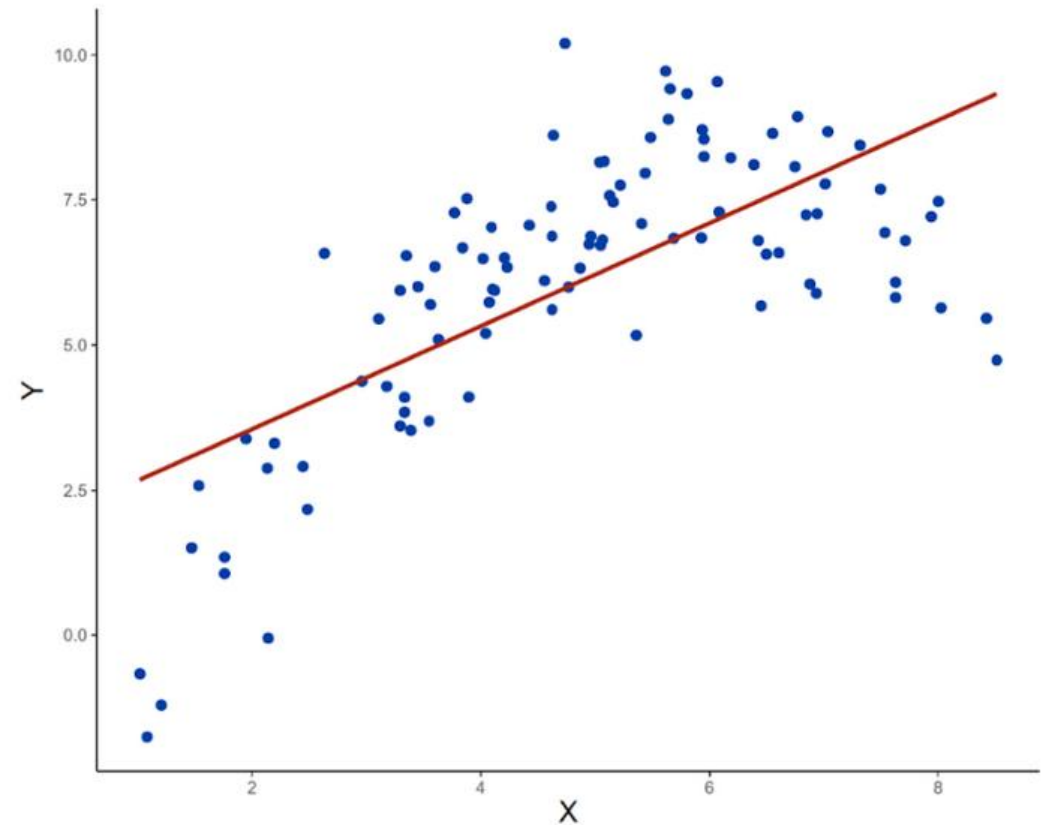
Model training

Learning algorithms for regression

- Attempting to fit a linear regression model

$$\hat{y} = \beta_0 + \beta_1 x$$

- Simple regression analysis fails in such conditions



Polynomial Regression: An Introduction: <https://builtin.com/machine-learning/polynomial-regression>

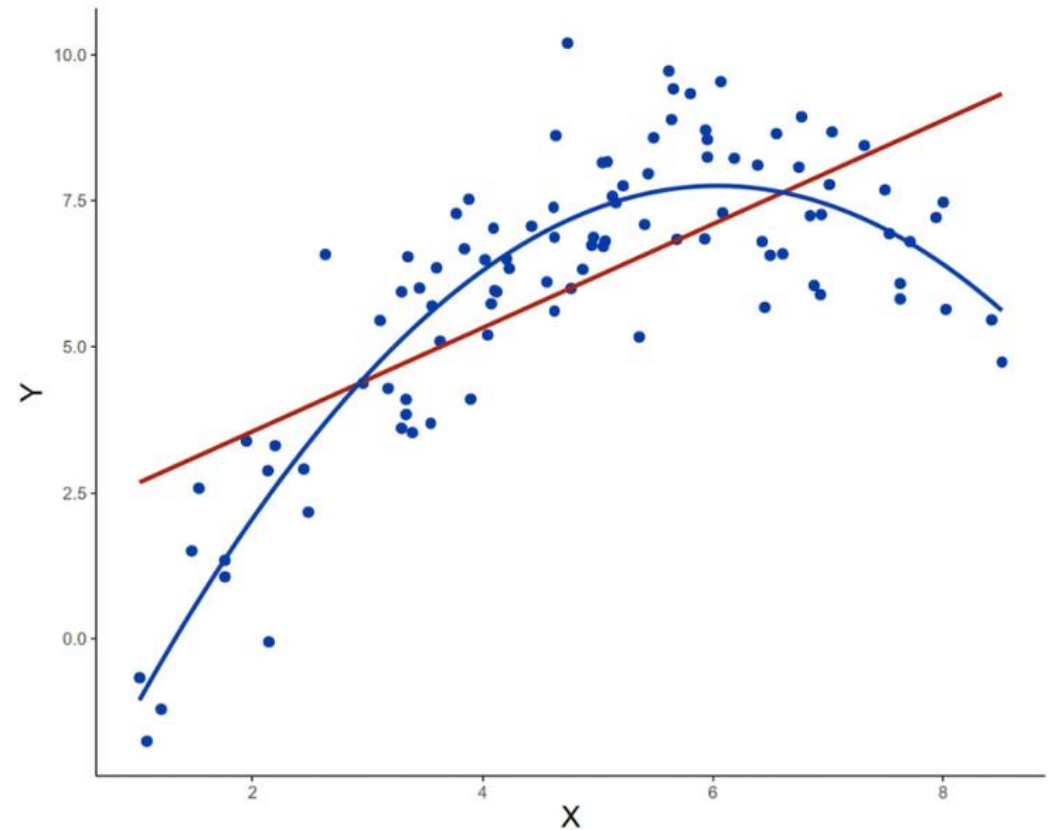
Model training

Learning algorithms for regression

- Fitting a **polynomial regression** model

$$\hat{y} = \beta_0 + \beta_1 x + \beta_2 x^2$$

- The approximation is much better



Polynomial Regression: An Introduction: <https://builtin.com/machine-learning/polynomial-regression>

Model training

Learning algorithms for regression

- Generalizing a **polynomial regression model** is:

$$\hat{y} = \beta_0 + \beta_1 x + \beta_2 x^2 + \dots + \beta_h x^h$$

- Where h is the **degree** of the polynomial
 - $h=2$ is a quadratic regression model
 - $h=3$ is a cubic regression model
 - ...
- Many learning algorithms for classification also have a counterpart for regression (e.g, decision trees, support vector for regression, kNN, neural networks, etc.)

Model training

Case study 1 – Titanic Survival Prediction



- **Dataset:** [Dataset \(Kaggle\): Titanic — Machine Learning from Disaster](#)
- **Goal:** Predict passenger survival after the Titanic sinking (binary classification)
- **Data:** Demographics and travel info (e.g., Sex, Age, Pclass, Fare, Embarked)
- **Challenges:** Missing values (Age, Cabin), small dataset, potential leakage via family/group tickets
- **Business framing:** Risk stratification and ranking (probability of survival) to illustrate classification basics

Model training

Case study 1 – Titanic Survival Prediction



- **Preprocessing:** Impute Age, handle missing Embarked; encode categoricals; normalize numeric features
- **Feature engineering:** FamilySize, IsAlone, Title (from Name), ticket group signals; bin Fare/Age if helpful
- **Model:** Logistic Regression (probability-based, discriminative) for interpretability or Decision Tree
- **Evaluation:** Stratified CV; metrics: Accuracy, ROC-AUC

Model training

Case study 2 – House Prices



- **Dataset:** [Dataset \(Kaggle\): House Prices — Advanced Regression Techniques](#)
- **Goal:** Predict residential sale price
- **Data:** Mix of numeric/ordinal/categorical variables (quality grades, area, neighborhood, garage/basement, etc.)
- **Challenges:** Many missing values; correlated features; outliers
- **Business framing:** Automated valuation models (AVM) and price sensitivity analysis

Model training

Case study 2 – House Prices



- **Preprocessing:** Target log-transform; impute domain-aware missing values; one-hot/ordinal encode; fix skewed vars
- **Model:** Gradient Boosting (e.g., XGBoost/LightGBM), optimization-based; strong on tabular with non-linearities
- **Evaluation:** K-fold CV with RMSE on log price; compare to linear baselines (Ridge/Lasso) for interpretability trade-offs

Model training

Case study 3 – Telco Customer Churn



- **Dataset:** [Dataset \(Kaggle\): Telco Customer Churn](#)
- **Goal:** Predict if a telecom customer will churn next month (binary classification)
- **Data:** Demographics, contract type, tenure, payment method, and subscribed services
- **Challenges:** Class imbalance; many categorical features; need for actionable insights for retention
- **Business framing:** Identify at-risk segments to trigger targeted offers (cost-sensitive decisions) – retention programs

Model training

Case study 3 – Telco Customer Churn



- **Preprocessing:** Clean TotalCharges; encode categoricals; treat imbalance with class weights or SMOTE
- **Model:** Decision Tree (search-based) or C5.0 for explicit, human-readable rules for operations teams
- **Evaluation:** Prioritize Recall/ROC-AUC and cost-aware metrics; calibrate thresholds by retention budget

Model training

Case study 4 – Credit Card Fraud



- **Dataset:** [Dataset \(Kaggle\): Credit Card Fraud Detection \(MLG-ULB\)](#)
- **Goal:** Detect fraudulent transactions in highly imbalanced data (<0.2% fraud)
- **Data:** PCA-transformed features (V1–V28), plus Time and Amount; European cardholders over two days
- **Challenges:** Extreme class imbalance; concept drift risk; high cost of false negatives
- **Business framing:** Real-time triage; human-in-the-loop for high-risk alerts

Model training

Case study 4 – Credit Card Fraud



- **Preprocessing:** Scale Amount/Time; stratified partitions; consider undersampling/oversampling; tune decision threshold
- **Model:** Support Vector Machine (optimization-based, margin) or Gradient Boosting for strong separation
- **Evaluation:** Precision-Recall curves and PR-AUC (recommended); monitor recall at low FPR

Model training

Case study 5 – CIFAR-10 Object Recognition in images



- **Dataset:** [Dataset \(Kaggle\): CIFAR-10](#)
- **Goal:** Classify 32×32 RGB images into 10 object classes (airplane, cat, ship, etc.)
- **Data:** 50k train + 10k test images; low resolution with intra-class variability
- **Challenges:** Need for spatial feature extraction; overfitting without augmentation
- **Business framing:** Edge/embedded vision prototypes; benchmarking feature learning

Model training

Case study 5 – CIFAR-10 Object Recognition in images



- **Preprocessing:** Normalize pixels; strong data augmentation (flips/crops/color jitter); early stopping
- **Model:** Convolutional Neural Network (optimization-based, parametrized) for spatial hierarchies
- **Evaluation:** Top-1 accuracy; confusion matrix by class; monitor generalization gap

Model training

Data Science Project

- In the context of each project:
 - **Dataset** – What is the dataset? Where is it located?
 - **Goal** – What is the problem you are trying to address? Is it classification or regression?
 - **Data** – What are the independent variables? What's their type? Which is the target variable?
 - **Challenges** – Which are the issues that need to be addressed when tackling the problem?
 - **Business framing** – How will the model contribute to the business goals?
 - **Preprocessing** – How does the data need to be changed before training?
 - **Model** – What is the learning algorithm used to train a model? Why?
 - **Evaluation** – Which is the method and metrics used for evaluation of models?